This doc is not an accurate representation of the final requirement. Please read the requirement document given by the professor, this document is just to help us distribute the work and an initial draft of what the work looks like. You have to figure out and work around the additional areas according to the professor's document.

LINKEDINCLONE

Distributed Systems Class Project

3-Tier + Kafka + Agentic AI + Analytics + Performance

MASTER ENGINEERING DOCUMENT

Single Source of Truth — 6 Team Member Prompts

Due Dates:

API Document: April 7 | Presentation: April 28 | Final Demo: May 5

TABLE OF CONTENTS

Part 1: System Architecture & Shared Contract

Part 2: Team Assignments & Role Matrix

Part 3: Shared API Contract (ALL members must read)

Part 4: Kafka Event Standard (ALL members must read)

Part 5: Exception / Failure Mode Catalogue (ALL members must read)

MEMBER 1: Ayush - Backend Services + Deployment (AWS/Docker) + Git & CI/CD Config

MEMBER 2: Manav - Frontend Developer — React UI (LinkedIn-accurate)

MEMBER 3: Bhoomika - Agentic AI Service — FastAPI + LLM Orchestration

MEMBER 4: khushi - Kafka + Messaging + Connections Services

MEMBER 5: Naman & Centhur Analytics, Dashboards, Redis Caching, Performance

MEMBER 6: Sharan Data Engineering, Testing, Documentation, Presentation

MEMBER 7: Actual Integration and merge + unit testing + integration testing

Part 6: Integration Checklist & Merging Protocol

PART 1 — SYSTEM ARCHITECTURE & SHARED CONSTRAINTS

Tech Stack (non-negotiable across all members): React (frontend) · FastAPI (AI service) · Node.js/Express or FastAPI (core services) · MySQL (transactional) · MongoDB (events/logs/agent traces) · Apache Kafka (async messaging) · Redis (caching) · Docker (containerisation) · AWS ECS or Kubernetes (deployment).

Architecture Tiers

- **Tier 1 — Client:** React web app, pixel-accurate LinkedIn clone. Communicates via REST and WebSocket to Tier 2.
- **Tier 2 — Services + Kafka:** Profile Service, Job Service, Application Service, Messaging Service, Connection Service, Analytics/Events Service, AI Agent Service. Producers push to Kafka; consumers process async. All services expose REST with the shared endpoint contract below.
- **Tier 3 — Databases:** MySQL for transactional records (members, jobs, applications, connections). MongoDB for Kafka event logs, agent traces, message bodies, unstructured profile data.

Port / Service Map (Fixed — Everyone Use This)

Service	Port	Owner
React Frontend	3000	Member 2
Profile Service	8001	Member 1
Job Service	8002	Member 1
Application Service	8003	Member 1
Messaging Service	8004	Member 4
Connection Service	8005	Member 4
Analytics/Events Service	8006	Member 5
AI Agent Service (FastAPI)	8007	Member 3
Kafka Broker	9092	Member 4
Zookeeper	2181	Member 4
MySQL	3306	Member 1
MongoDB	27017	Member 1
Redis	6379	Member 5

PART 2 — TEAM ASSIGNMENTS & ROLE MATRIX

Member	Phase 1 (Now → W3)	Phase 2 (W4 → Final)	Color Code
Member 1 (Ayush)	GitHub repo owner: create repo + README + mono-repo structure. Backend: Profile, Job, Application REST services + MySQL + MongoDB schema + seed loader	Docker Compose + AWS ECS/K8s deployment + integration hardening + perf benchmarks	Blue
Member 2	React UI: Authentication, Member profile, Job search/list/detail, Apply flow (LinkedIn-accurate)	Recruiter dashboard UI, AI copilot panel, WebSocket notification panel, Presentation slides	Purple
Member 3	FastAPI AI service: Resume Parser, Job-Candidate Matcher, Hiring Assistant Supervisor + Kafka ai.requests/ai.results	Career Coach Agent, human-in-the-loop approval UI integration, AI metrics + evaluation report	Green
Member 4	Kafka broker setup, Messaging Service (threads+messages), Connection Service, Kafka topic definitions	End-to-end async workflow validation, consumer idempotency hardening, DLQ, failure testing	Orange
Member 5	Analytics/Events Service, Redis caching layer, event ingestion pipeline, recruiter/member dashboards	Performance benchmark suite (JMeter B/B+S/B+S+K), Redis impact charts, scalability report	Teal
Member 6	Dataset pipeline: Kaggle jobs + resumes → seed scripts, API document (deliverable A), DB schema diagrams	Test class (unit + integration), project write-up, presentation deck, demo script	Pink

---

GitHub Workflow (All Members Follow)

- **Week 0 (do first):** Member 6 creates the repo: "linkedinclone", pushes README.md with project overview, tech stack, member table, setup instructions. Sets up branch protection on main. All others fork or clone.
- **Branch convention:** feature/<member>/<short-description> e.g. feature/member1/profile-service
- **PR rule:** At least 1 review from another member before merge to main. CI must pass (lint + unit tests).
- **Mono-repo structure:** /frontend /services/profile /services/job /services/application /services/messaging /services/connection /services/analytics /services/ai-agent /infra /data /docs /tests

PART 3 — SHARED API CONTRACT (ALL MEMBERS READ)

CRITICAL: Every service must use POST for all endpoints (as specified). Request and response bodies must be JSON. All endpoints must return meaningful HTTP status codes (200, 201, 400, 404, 409, 500). All error responses must follow the standard error envelope.

Standard Response Envelopes

```
// SUCCESS
{ "success": true, "data": { ... }, "trace_id": "uuid" }

// ERROR
{ "success": false, "error": { "code": "DUPLICATE_EMAIL", "message": "...",
"details": {} }, "trace_id": "uuid" }
```

Profile Service — POST /members/* (port 8001)

Endpoint	Request Body	Response
POST /members/create	body: {first_name, last_name, email, phone, location, headline, about, skills[], experience[], education[], profile_photo_url}	201: {member_id, ...profile} | 409: DUPLICATE_EMAIL | 400: VALIDATION_ERROR
POST /members/get	body: {member_id}	200: full profile | 404: MEMBER_NOT_FOUND
POST /members/update	body: {member_id, fields_to_update...}	200: updated profile | 404: MEMBER_NOT_FOUND | 400: VALIDATION_ERROR
POST /members/delete	body: {member_id}	200: {deleted: true} | 404: MEMBER_NOT_FOUND
POST /members/search	body: {keyword?, skill?, location?, page, page_size}	200: {results:[], total, page}

Job Service — POST /jobs/* (port 8002)

Endpoint	Request Body	Response
POST /jobs/create	body: {company_id, recruiter_id, title, description, seniority_level, employment_type, location, remote_type, skills_required[], salary_range?, status}	201: {job_id, ...} | 400: VALIDATION_ERROR | 404: RECRUITER_NOT_FOUND
POST /jobs/get	body: {job_id}	200: full job | 404: JOB_NOT_FOUND
POST /jobs/update	body: {job_id, fields...}	200: updated | 404: JOB_NOT_FOUND
POST /jobs/search	body: {keyword?, location?, employment_type?, industry?, remote_type?, page, page_size}	200: {results:[], total, page}
POST /jobs/close	body: {job_id}	200: {status:'closed'} | 404: JOB_NOT_FOUND | 409: ALREADY_CLOSED
POST /jobs/byRecruiter	body: {recruiter_id, page, page_size}	200: {results:[], total}

Application Service — POST /applications/* (port 8003)

Endpoint	Request Body	Response
POST /applications/submit	body: {job_id, member_id, resume_url?, resume_text?, cover_letter?, answers?}	201: {application_id} | 409: DUPLICATE_APPLICATION | 409: JOB_CLOSED | 404: JOB/MEMBER_NOT_FOUND
POST /applications/get	body: {application_id}	200: full application | 404: NOT_FOUND
POST /applications/byJob	body: {job_id, page, page_size}	200: {results:[], total} (recruiter view)
POST /applications/byMember	body: {member_id, page, page_size}	200: {results:[], total} (member view)
POST /applications/updateStatus	body: {application_id, status: submitted|reviewing|interview|offer|rejected, note?}	200: {updated: true} | 400: INVALID_STATUS_TRANSITION
POST /applications/addNote	body: {application_id, recruiter_id, note_text}	200: {note_id} | 404: APPLICATION_NOT_FOUND

Messaging Service — POST /threads/* + /messages/* (port 8004)

Endpoint	Request Body	Response
POST /threads/open	{participant_ids: [id1, id2]}	201: {thread_id} | 409: THREAD_EXISTS
POST /threads/get	{thread_id}	200: thread meta | 404
POST /messages/list	{thread_id, page, page_size}	200: {messages:[], total}
POST /messages/send	{thread_id, sender_id, message_text}	201: {message_id} | 404: THREAD_NOT_FOUND | 503: KAFKA_UNAVAILABLE (retry)
POST /threads/byUser	{user_id, page, page_size}	200: {threads:[], total}

Connection Service — POST /connections/* (port 8005)

Endpoint	Request Body	Response
POST /connections/request	{requester_id, receiver_id}	201: {request_id} | 409: ALREADY_CONNECTED / PENDING_REQUEST
POST /connections/accept	{request_id}	200: {connected: true} | 404: REQUEST_NOT_FOUND
POST /connections/reject	{request_id}	200: {rejected: true} | 404: REQUEST_NOT_FOUND
POST /connections/list	{user_id, page, page_size}	200: {connections:[], total}
POST /connections/mutual	{user_id, other_id}	200: {mutual_connections:[], count} (extra credit)

Analytics/Events Service — POST /events/* + /analytics/* (port 8006)

Endpoint	Request Body	Response

POST /events/ingest	{event_type, actor_id, entity_type, entity_id, payload, trace_id}	202: {accepted: true}
POST /analytics/jobs/top	{metric: applications|views|saves, window_days, limit}	200: {jobs:[{job_id, title, count}]}
POST /analytics/funnel	{job_id, window_days}	200: {view, save, apply_start, submit, rates:{}}
POST /analytics/geo	{job_id, window_days}	200: {cities:[{city, state, count}]}
POST /analytics/member/dashboard	{member_id, window_days}	200: {profile_views:[], application_status_breakdown:{}}

AI Agent Service — POST /ai/* + WebSocket (port 8007 FastAPI)

Endpoint	Params/Body	Response
POST /ai/request	{job_id, recruiter_id, task_type: shortlist|match|parse|coach}	202: {task_id, trace_id} | 400: INVALID_TASK
POST /ai/status	{task_id}	200: {status: pending|running|completed|failed, steps:[], result?}
POST /ai/approve	{task_id, action: approve|edit|reject, edited_content?}	200: {actioned: true}
WS /ai/stream/{task_id}	WebSocket — streams step progress events to UI	JSON frames: {step, status, partial_result, trace_id}

PART 4 — KAFKA EVENT STANDARD (ALL MEMBERS READ)

Every Kafka message MUST use this exact JSON envelope. No exceptions.

```
{
  "event_type": "job.viewed | job.saved | application.submitted | message.sent |
                 connection.requested | ai.requested | ai.completed",
  "trace_id":   "uuid-v4  — stays same across entire multi-step workflow",
  "timestamp":  "ISO-8601 e.g. 2025-04-07T10:30:00.000Z",
  "actor_id":   "member_id or recruiter_id",
  "entity": {
    "entity_type": "job | application | thread | connection | ai_task",
    "entity_id":   "..."
  },
  "payload": { "domain_specific_fields": "..." },
  "idempotency_key": "uuid-v4 or sha256-hash-of-key-fields"
}
```

Kafka Topics (Required)

Topic Name	Producer	Consumer	Owner
`job.viewed`	Job Service / Frontend	Analytics Service	Member 4/5
`job.saved`	Job Service / Frontend	Analytics Service	Member 4/5
`application.submitted`	Application Service	Analytics + AI Agent	Member 1/4
`application.status.updated`	Application Service	Frontend (notify)	Member 1/4
`message.sent`	Messaging Service	Messaging Consumer	Member 4
`connection.requested`	Connection Service	Connection Consumer	Member 4
`connection.accepted`	Connection Service	Analytics	Member 4
`ai.requests`	AI Agent Service	Hiring Assistant Supervisor	Member 3
`ai.results`	Hiring Assistant Supervisor	AI Agent Service + Frontend	Member 3

Idempotency Rules

- **Every consumer** must check the idempotency_key against a processed_events table or Redis set before performing any DB write.
- **If already processed:** log and skip — do not return error to Kafka (do not re-queue).
- **Consumer crash recovery:** Kafka at-least-once delivery means the same event can arrive multiple times. Idempotent processing makes this safe.
- **DLQ (Dead Letter Queue):** After 3 failed retries, route to <topic>.dlq. Member 4 provisions these.

PART 5 — EXCEPTION & FAILURE MODE CATALOGUE (ALL MEMBERS READ)

REQUIRED: Every failure mode below must be handled. Graders will explicitly test these. Each member must handle the failures relevant to their service.

Failure Mode	Error Code	HTTP Status	Handling Required	Owner
Duplicate email on member create	`DUPLICATE_EMAIL`	409	Check before insert; return error with existing member_id hint	Member 1
Duplicate application same job+member	`DUPLICATE_APPLICATION`	409	Unique constraint (job_id, member_id). Return existing application_id	Member 1
Apply to a closed job	`JOB_CLOSED`	409	Check job.status before insert. Atomic read+write in transaction	Member 1
Message send + Kafka unavailable	`KAFKA_UNAVAILABLE`	503	Retry up to 3x with exponential backoff. Fallback: store in outbox table. Return 503 with retry-after header	Member 4
Kafka consumer crash / restart	—	—	Idempotency key prevents duplicate writes. Kafka offset only committed after successful DB write. DLQ after 3 failures	Member 4
Partial multi-step failure (e.g. create member + emit Kafka)	`PARTIAL_FAILURE`	500	DB transaction wraps all writes. If Kafka emit fails, roll back DB. Return 500 with trace_id for debugging	Member 1
AI task timeout / LLM failure	`AI_TASK_FAILED`	—	Set 30s timeout on AI skill calls. On timeout emit ai.results with status:failed, reason:timeout. UI shows error state	Member 3
Member/Job not found on any GET	`NOT_FOUND`	404	All GET endpoints return 404 with specific entity type in error code	Member 1
Invalid status transition on application	`INVALID_STATUS_TRANSITION`	400	State machine: submitted→reviewing→interview→offer|rejected. Validate transition; return 400 with current + allowed next states	Member 1
Redis cache miss / Redis down	—	—	Fall through to DB on cache miss. If Redis completely down, degrade gracefully (log warning, skip caching)	Member 5
Concurrent connection request (both sides)	`ALREADY_CONNECTED`	409	Unique constraint on (min(a,b), max(a,b)) pair. Check before insert	Member 4

MEMBER 1: Ayush

Domain: Backend Services + AWS/Docker Deployment

Phase 1 → Profile + Job + Application Services, MySQL + MongoDB | Phase 2 → Docker Compose + AWS ECS/K8s + Integration + Benchmarks

GIVE THIS PROMPT + THE PROJECT DESCRIPTION DOC TO YOUR CODING AGENT. Your agent has full freedom to improve internals, fill gaps, and enhance quality — but must never deviate from the shared API contract (Part 3), Kafka envelope (Part 4), or failure handling (Part 5).

Context & Scope

You are building the core backend services for a LinkedIn-like distributed system. This is a class project for Distributed Systems at SJSU. Your work is the foundation all other team members depend on. Build robustly, document every endpoint, and prioritize correctness over cleverness.

PHASE 1: Core Backend Services (W0–W3)

Step 1: Repository Creation & README — YOU OWN THIS (Do First — W0)

1. **Create GitHub repository:** "linkedinclone" — you are the repo owner. Initialize with a README.md containing: project title, description (LinkedIn-like distributed system), tech stack table, team member table (Member 1–6 with roles), architecture diagram placeholder, setup instructions skeleton, due dates (API Doc: Apr 7, Presentation: Apr 28, Demo: May 5). Push to main. Share the repo link with all 5 team members immediately.
2. **Branch protection:** Enable branch protection on main: require PR review (min 1 approval) + CI pass before merge.
3. **Mono-repo structure:** Create folders: /frontend /services/profile /services/job /services/application /services/messaging /services/connection /services/analytics /services/ai-agent /infra /data /docs /tests. Each service folder gets its own package.json or requirements.txt and a Dockerfile.
4. **Docker Compose (infra):** Create /infra/docker-compose.yml that starts: MySQL (port 3306), MongoDB (port 27017), Redis (port 6379), Zookeeper (port 2181), Kafka (port 9092). Use named volumes. Add healthchecks. This is the shared dev environment for ALL team members.
5. **Environment config:** Create .env.example with all required variables: DB_HOST, DB_PORT, DB_USER, DB_PASS, DB_NAME, MONGO_URI, REDIS_HOST, REDIS_PORT, KAFKA_BROKERS. Never commit .env. Add to .gitignore.

Step 2: MySQL Schema (W0–W1)

Create /data/schema.sql with the following tables. Run CREATE DATABASE IF NOT EXISTS linkedinclone; USE linkedinclone; at top.

```
-- MEMBERS
CREATE TABLE members (
  member_id      VARCHAR(36) PRIMARY KEY DEFAULT (UUID()),
  first_name     VARCHAR(100) NOT NULL,
  last_name      VARCHAR(100) NOT NULL,
  email          VARCHAR(255) NOT NULL UNIQUE,
  phone          VARCHAR(20),
  location       VARCHAR(255),
  headline       VARCHAR(500),
  about          TEXT,
```

```sql
  profile_photo_url VARCHAR(500),
  connections_count INT DEFAULT 0,
  created_at     DATETIME DEFAULT CURRENT_TIMESTAMP,
  updated_at     DATETIME DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
);
CREATE INDEX idx_members_email ON members(email);
CREATE FULLTEXT INDEX idx_members_ft ON members(first_name, last_name, headline,
about);

-- RECRUITERS
CREATE TABLE recruiters (
  recruiter_id   VARCHAR(36) PRIMARY KEY DEFAULT (UUID()),
  company_id     VARCHAR(36) NOT NULL,
  name           VARCHAR(200) NOT NULL,
  email          VARCHAR(255) NOT NULL UNIQUE,
  phone          VARCHAR(20),
  company_name   VARCHAR(300) NOT NULL,
  company_industry VARCHAR(200),
  company_size   VARCHAR(100),
  role           VARCHAR(100) DEFAULT 'recruiter',
  access_level   ENUM('recruiter','admin') DEFAULT 'recruiter',
  created_at     DATETIME DEFAULT CURRENT_TIMESTAMP
);

-- JOB POSTINGS
CREATE TABLE jobs (
  job_id         VARCHAR(36) PRIMARY KEY DEFAULT (UUID()),
  company_id     VARCHAR(36) NOT NULL,
  recruiter_id   VARCHAR(36) NOT NULL,
  title          VARCHAR(300) NOT NULL,
  description    TEXT NOT NULL,
  seniority_level
ENUM('internship','entry','associate','mid','senior','director','executive'),
  employment_type
ENUM('full_time','part_time','contract','temporary','volunteer','internship'),
  location       VARCHAR(255),
  remote_type    ENUM('onsite','remote','hybrid') DEFAULT 'onsite',
  salary_range   VARCHAR(100),
  status         ENUM('open','closed') DEFAULT 'open',
  posted_datetime DATETIME DEFAULT CURRENT_TIMESTAMP,
  views_count    INT DEFAULT 0,
  applicants_count INT DEFAULT 0,
  FOREIGN KEY (recruiter_id) REFERENCES recruiters(recruiter_id)
);
CREATE INDEX idx_jobs_status ON jobs(status);
CREATE INDEX idx_jobs_recruiter ON jobs(recruiter_id);
CREATE FULLTEXT INDEX idx_jobs_ft ON jobs(title, description, location);

-- JOB SKILLS (many-to-many)
CREATE TABLE job_skills (
  job_id    VARCHAR(36), skill VARCHAR(200),
  PRIMARY KEY(job_id, skill), FOREIGN KEY(job_id) REFERENCES jobs(job_id)
);

-- APPLICATIONS
CREATE TABLE applications (
  application_id  VARCHAR(36) PRIMARY KEY DEFAULT (UUID()),
  job_id          VARCHAR(36) NOT NULL,
  member_id       VARCHAR(36) NOT NULL,
  resume_url      VARCHAR(500),
  resume_text     TEXT,
  cover_letter    TEXT,
  application_datetime DATETIME DEFAULT CURRENT_TIMESTAMP,
```

```
  status  ENUM('submitted','reviewing','interview','offer','rejected') DEFAULT
'submitted',
  UNIQUE KEY uk_job_member (job_id, member_id),
  FOREIGN KEY (job_id) REFERENCES jobs(job_id),
  FOREIGN KEY (member_id) REFERENCES members(member_id)
);

-- APPLICATION NOTES
CREATE TABLE application_notes (
  note_id        VARCHAR(36) PRIMARY KEY DEFAULT (UUID()),
  application_id VARCHAR(36) NOT NULL,
  recruiter_id   VARCHAR(36) NOT NULL,
  note_text      TEXT NOT NULL,
  created_at     DATETIME DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (application_id) REFERENCES applications(application_id)
);

-- CONNECTIONS
CREATE TABLE connections (
  connection_id  VARCHAR(36) PRIMARY KEY DEFAULT (UUID()),
  user_a         VARCHAR(36) NOT NULL,
  user_b         VARCHAR(36) NOT NULL,
  status         ENUM('pending','accepted','rejected') DEFAULT 'pending',
  requested_by   VARCHAR(36) NOT NULL,
  created_at     DATETIME DEFAULT CURRENT_TIMESTAMP,
  UNIQUE KEY uk_connection_pair (user_a, user_b)
);

-- MESSAGING THREADS
CREATE TABLE threads (
  thread_id   VARCHAR(36) PRIMARY KEY DEFAULT (UUID()),
  created_at  DATETIME DEFAULT CURRENT_TIMESTAMP
);
CREATE TABLE thread_participants (
  thread_id VARCHAR(36), user_id VARCHAR(36),
  PRIMARY KEY(thread_id, user_id)
);
CREATE TABLE messages (
  message_id   VARCHAR(36) PRIMARY KEY DEFAULT (UUID()),
  thread_id    VARCHAR(36) NOT NULL,
  sender_id    VARCHAR(36) NOT NULL,
  message_text TEXT NOT NULL,
  sent_at      DATETIME DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (thread_id) REFERENCES threads(thread_id)
);
CREATE INDEX idx_messages_thread ON messages(thread_id, sent_at);

-- MEMBER SKILLS, EXPERIENCE, EDUCATION (structured)
CREATE TABLE member_skills (member_id VARCHAR(36), skill VARCHAR(200), PRIMARY
KEY(member_id, skill));
CREATE TABLE member_experience (
  exp_id VARCHAR(36) PRIMARY KEY DEFAULT(UUID()), member_id VARCHAR(36),
  company VARCHAR(300), title VARCHAR(300), start_date DATE, end_date DATE,
  description TEXT, is_current BOOLEAN DEFAULT FALSE
);
CREATE TABLE member_education (
  edu_id VARCHAR(36) PRIMARY KEY DEFAULT(UUID()), member_id VARCHAR(36),
  institution VARCHAR(300), degree VARCHAR(200), field VARCHAR(200),
  start_year INT, end_year INT
);

-- PROCESSED EVENTS (idempotency)
CREATE TABLE processed_events (
  idempotency_key VARCHAR(255) PRIMARY KEY,
```

```
  processed_at    DATETIME DEFAULT CURRENT_TIMESTAMP
);
```

Step 3: MongoDB Collections (W0–W1)

Create /data/mongo_setup.js. MongoDB stores: event logs, agent traces, profile sections (resume text), message bodies (optional mirror for full-text search).

```
// Run with: mongosh < mongo_setup.js
use linkedinclone;

// Events / Analytics log (append-only, from Kafka consumers)
db.createCollection('events', { capped: false });
db.events.createIndex({ event_type: 1, timestamp: -1 });
db.events.createIndex({ actor_id: 1, timestamp: -1 });
db.events.createIndex({ 'entity.entity_id': 1 });
db.events.createIndex({ idempotency_key: 1 }, { unique: true });

// AI agent task traces
db.createCollection('ai_traces');
db.ai_traces.createIndex({ task_id: 1 }, { unique: true });
db.ai_traces.createIndex({ trace_id: 1 });
db.ai_traces.createIndex({ status: 1, created_at: -1 });

// Profile resumes (raw text from uploaded PDFs)
db.createCollection('resumes');
db.resumes.createIndex({ member_id: 1 });

// Profile views tracking (time-series-style)
db.createCollection('profile_views');
db.profile_views.createIndex({ member_id: 1, viewed_at: -1 });
```

Step 4: Service Implementation — Feature by Feature (W1–W3)

Build one feature at a time, in this order. Test each with curl/Postman before moving on.

6. **Profile Service (port 8001) — build in order:** POST /members/create → /members/get → /members/update → /members/delete → /members/search. Use Express.js or FastAPI. Connect to MySQL. Validate all required fields. Emit Kafka event on create/update/delete (if Kafka is up — fail gracefully if not, use outbox pattern).
7. **Job Service (port 8002) — build in order:** /jobs/create → /jobs/get → /jobs/update → /jobs/search → /jobs/close → /jobs/byRecruiter. FULLTEXT search on title+description+location. views_count incremented on every /jobs/get (Kafka event job.viewed). applicants_count updated by Application Service via Kafka consumer.
8. **Application Service (port 8003) — build in order:** /applications/submit → /applications/get → /applications/byJob → /applications/byMember → /applications/updateStatus → /applications/addNote. Check for duplicate application (409). Check job is open (409). Wrap submit in MySQL transaction: insert application + increment jobs.applicants_count + emit Kafka application.submitted. Roll back fully on any failure.
9. **Health check endpoint:** Every service must expose GET /health → {status:'ok', service:'profile', db:'connected', kafka:'connected'}. Used by Docker healthchecks and AWS ALB.

Step 5: Kafka Producer Utility (Shared)

Create /services/shared/kafka.js (or kafka.py). Wrap kafkajs (Node) or confluent-kafka-python. Implement: connect(), produce(topic, envelope), disconnect(). The envelope builder must take (event_type, actor_id,

entity_type, entity_id, payload) and auto-generate trace_id and idempotency_key. Export as a reusable module all services import.

Implement outbox pattern: if Kafka is unavailable, store the message in MySQL table outbox_events (columns: id, topic, envelope JSON, created_at, sent BOOLEAN). A background poller every 5s retries unsent outbox messages. This ensures no event is lost.

PHASE 2: Deployment (W5–W6)

Step 6: Dockerize All Services

10. **Each service:** Create a Dockerfile with multi-stage build. Final image: node:20-alpine or python:3.11-slim. Expose correct port. CMD runs the app. Add .dockerignore.
11. **Update docker-compose.yml:** Add all 7 services (profile, job, application, messaging, connection, analytics, ai-agent). Wire environment variables from .env. Set depends_on: mysql, mongodb, redis, kafka. Add restart: unless-stopped.
12. **docker-compose up:** Test that all services start, pass healthchecks, and are reachable at their ports. Fix any startup order issues.

Step 7: AWS Deployment (ECS Fargate recommended, or Kubernetes)

13. **ECR:** Push all Docker images to Amazon ECR. Tag with git SHA for traceability.
14. **ECS Task Definitions:** One per service. Set CPU/memory (512 CPU / 1GB RAM per service minimum). Pass secrets via AWS Secrets Manager (DB creds, Kafka creds). Set health check paths (/health).
15. **ECS Services:** Deploy each task definition. Use Application Load Balancer for frontend and API gateway. Configure target groups per service port.
16. **RDS MySQL:** Use AWS RDS MySQL t3.micro for class project. Run schema.sql on startup. Enable automated backups.
17. **DocumentDB or Atlas:** MongoDB — use MongoDB Atlas free tier for class project. Store connection string in Secrets Manager.
18. **ElastiCache Redis:** Use AWS ElastiCache Redis t3.micro or deploy Redis in ECS.
19. **MSK Kafka:** Use AWS MSK (Managed Streaming for Kafka) t3.small for class project, or deploy Kafka in ECS with Zookeeper.
20. **Deployment comparison:** Document 1-service vs multi-replica latency/throughput for the presentation (Member 5 runs JMeter, you provide the deployment config for both scenarios).

Step 8: Performance Benchmark Support

Work with Member 5 to ensure the benchmark infrastructure is correct:

- Expose /benchmark/seed endpoint that loads 10,000 members + 10,000 jobs + proportional applications in one call (for JMeter setup).
- Confirm Redis caching is correctly wired to Profile and Job services (Member 5 implements the caching logic — you provide the DB layer hooks).
- Confirm Kafka is producing/consuming correctly for the benchmarks.

Quality Requirements (Apply Throughout)

- **Input validation:** All POST bodies must be validated (required fields, type checks, length limits). Use Joi/Zod (Node) or Pydantic (Python). Return 400 with field-level errors.

- **Transactions:** Any operation that writes to more than one table must use a database transaction with rollback on failure.
- **Consistent error format:** { "success": false, "error": { "code": "...", "message": "...", "details": {} }, "trace_id": "..." }
- **Logging:** Use structured JSON logging (winston/morgan for Node, loguru for Python). Log every request: method, path, duration_ms, status, trace_id.
- **No exposed secrets:** All credentials via environment variables. Validate on startup — crash immediately if required env vars are missing.

MEMBER 2: Frontend Developer

Domain: React UI — LinkedIn-Accurate Clone

Phase 1 → Auth + Member Profile + Job Search + Apply Flow | Phase 2 → Recruiter Dashboard + AI Copilot Panel + Presentation Slides

GIVE THIS PROMPT + THE PROJECT DESCRIPTION DOC TO YOUR CODING AGENT.

Context & Scope

You are building the frontend for a LinkedIn-like distributed system. The UI must be pixel-accurate to the latest LinkedIn (2024/2025 design). Reference https://www.linkedin.com for visual accuracy. Use React with Tailwind CSS. The backend API contract is fixed (Part 3). Use Axios for HTTP calls. All Kafka-driven notifications arrive via WebSocket from the AI Agent Service (port 8007).

Tech Stack (Required)

- React 18 + Vite
- Tailwind CSS (LinkedIn color palette: #0A66C2 primary, #1B1F23 text, #EAECEF background, #FFFFFF cards)
- React Router v6 for navigation
- Axios for API calls with global interceptors
- Recharts for analytics dashboard graphs
- React Query (@tanstack/query) for server state + cache
- Zustand for global auth state
- React Hook Form + Zod for form validation

PHASE 1: Core UI (W1–W3) — One Feature at a Time

Step 1: Project Setup

21. Create Vite React app. Configure Tailwind. Set up React Router with route structure. Create api/client.js with Axios base URL configured via VITE_API_BASE_URL env var.
22. Create global API interceptors: attach auth token to every request. Handle 401 → redirect to login. Handle 503 (Kafka unavailable) → show retry toast.
23. Set up LinkedIn-accurate design system: colors, typography (LinkedIn uses -apple-system, BlinkMacSystemFont, Segoe UI, Roboto), spacing, card shadows, button variants.

Step 2: Authentication Pages

24. Login page: LinkedIn-accurate layout. Email + password form. On success: store JWT in Zustand + localStorage. Route: /login.
25. Register page (member): All member fields from Part 3 schema. Skills multi-select input. Step form (basic info → experience → education → skills). Route: /register.
26. Register page (recruiter): recruiter fields. Route: /register/recruiter.
27. Auth guard: HOC or route wrapper that redirects unauthenticated users to /login.

Step 3: Member Feed & Profile

28. LinkedIn-accurate header: Logo left, search bar center, nav icons right (Home, My Network, Jobs, Messaging, Notifications, Me). Exactly as LinkedIn 2024.
29. Member profile page (/profile/:member_id): Photo + name + headline + location + about. Experience timeline. Education. Skills chips. Connection count. Profile views widget (calls /analytics/member/dashboard). Connect / Message / More buttons.
30. Edit profile modal: Inline edit for all fields. Calls POST /members/update. Show success toast.

Step 4: Job Search & Listings

31. Job search page (/jobs): Search bar + filters sidebar (location, employment_type, remote_type, industry). Calls POST /jobs/search with debounce 300ms.
32. Job results list: LinkedIn-accurate job card (title, company, location, posted time, Easy Apply badge if applicable, Save button). Clicking a card loads job detail in right panel.
33. Job detail panel: Full description, required skills chips, salary range, applicants count, Save button (POST /events/ingest with job.saved), Apply button.
34. Save job: Calls /events/ingest {event_type: 'job.saved', ...}. Updates local state immediately (optimistic update).

Step 5: Application Flow

35. Apply modal (standard template): Pre-fill member profile fields. Resume upload (stores URL). Cover letter textarea (optional). Submit → POST /applications/submit.
36. Handle errors: 409 DUPLICATE_APPLICATION → 'You have already applied to this job.' 409 JOB_CLOSED → 'This position is no longer accepting applications.'
37. Applications tracker (/my-applications): List all applications for logged-in member (POST /applications/byMember). Status badges with LinkedIn colors (submitted: grey, reviewing: blue, interview: green, offer: gold, rejected: red).

Step 6: Messaging

38. Messaging page (/messaging): Left panel: thread list (POST /threads/byUser). Right panel: message bubbles. LinkedIn-accurate messenger layout.
39. Send message: POST /messages/send. Optimistic UI — append bubble immediately, reconcile on response.
40. Real-time: Connect WebSocket to ws://localhost:8007/ai/stream (reuse for notifications). Show incoming message notification badge.

Step 7: Connections

41. My Network page (/network): People you may know (from /members/search random sample). Connection requests pending. Your connections list.
42. Send/accept/reject connection requests via Connection Service endpoints.

PHASE 2: Recruiter Dashboard + AI Copilot (W4–W5)

Step 8: Recruiter Dashboard

43. Recruiter job management (/recruiter/jobs): Create/edit/close job postings. List with applicant count.
44. Applicants view (/recruiter/jobs/:job_id/applicants): List of applications (POST /applications/byJob). Click to view full profile + resume. Status dropdown (POST /applications/updateStatus). Add note (POST /applications/addNote).

45. Analytics charts (from /analytics/* endpoints using Recharts): Top 10 jobs by applications (bar chart). City-wise distribution (horizontal bar or map). Top 5 lowest-traction jobs (bar chart). Clicks per job (line chart). Saved jobs per day (line chart). All charts must be interactive with tooltips.

Step 9: AI Copilot Panel

46. AI Copilot button on job detail (recruiter view): Opens right drawer panel.
47. Trigger AI workflow: POST /ai/request {job_id, recruiter_id, task_type: 'shortlist'}. Display task_id + polling spinner.
48. WebSocket stream: Connect to ws://[host]:8007/ai/stream/{task_id}. Show step progress: 'Parsing resumes...', 'Computing match scores...', 'Generating outreach draft...'. Each step shows as a timeline card.
49. Human-in-the-loop: When AI returns shortlist + outreach draft, show approval UI. Approve / Edit / Reject buttons per candidate + message draft. POST /ai/approve with action.
50. Handle AI failure: If stream receives status:failed, show error state with retry button.

Step 10: Member Analytics Dashboard

51. Profile views chart: Line chart (last 30 days). Data from /analytics/member/dashboard.
52. Application status breakdown: Donut chart. Shows count per status.

Quality Requirements

- **Pixel accuracy:** Compare every page against real LinkedIn 2024 in a browser side-by-side. Font sizes, card padding, button styles, color values must match.
- **Loading states:** Every API call shows skeleton loaders (LinkedIn-accurate pulsing grey bars).
- **Error states:** Every API error shows a toast notification (top-right, LinkedIn style).
- **Responsive:** Works at 1440px desktop (primary) and 768px tablet.
- **Accessibility:** All buttons/inputs have aria-labels. Keyboard navigable.
- **Environment:** VITE_API_BASE_URL=http://localhost (dev) → configured via env for AWS deployment.

MEMBER 3: AI Agent Engineer

Domain: FastAPI Agentic AI Service — LLM Orchestration + Kafka

Phase 1 → Resume Parser + Job Matcher + Hiring Assistant Supervisor | Phase 2 → Career Coach Agent + AI Metrics + Evaluation Report

GIVE THIS PROMPT + THE PROJECT DESCRIPTION DOC TO YOUR CODING AGENT.

Context & Scope

You are building the Agentic AI layer — a FastAPI microservice that orchestrates multiple AI skills to support LinkedIn-style recruiting workflows. The system behaves like a recruiter copilot. It must: (1) accept tasks via REST, (2) coordinate AI skills via Kafka (ai.requests / ai.results topics), (3) stream progress to the frontend via WebSockets, (4) require human approval before any recruiter-facing action is taken.

Tech Stack (Required)

- Python 3.11 + FastAPI + Uvicorn
- WebSockets (FastAPI native)
- confluent-kafka-python (Kafka producer + consumer)
- MongoDB (pymongo) — store all agent traces, task states, step results
- OpenAI API or local LLM (ollama) — for resume parsing, matching, outreach drafting
- sentence-transformers (HuggingFace) — for embedding-based job-candidate matching
- Pydantic v2 — all request/response models

PHASE 1: Core AI Services (W2–W4) — One Skill at a Time

Step 1: FastAPI Project Setup

53. Create /services/ai-agent/ with FastAPI app. Expose port 8007. Configure CORS for frontend origin.
54. MongoDB connection: pymongo client pointing to linkedinclone.ai_traces collection.
55. Kafka producer: connect to broker, produce to ai.requests and ai.results.
56. Kafka consumer thread: background thread consuming from ai.results (for task status updates).
57. WebSocket manager class: ConnectionManager that maintains a dict of {task_id: [WebSocket]}. Methods: connect(task_id, ws), disconnect(task_id, ws), broadcast(task_id, message).

Step 2: Skill 1 — Resume Parser

Input: {resume_text: str, member_id: str} Output: {skills:[], years_experience:int, education:[], summary:str}

58. POST /ai/skills/parse-resume: Accept resume text. Call LLM with structured prompt to extract: skills list, total years of experience, education (degree, institution, year), summary sentence.
59. Use Pydantic model for output. Validate that required fields are present. If LLM returns malformed JSON, retry once, then return partial result with error flag.
60. Store result in MongoDB ai_traces with task_id, step:'resume_parse', status:'completed', result:{...}, timestamp.

Step 3: Skill 2 — Job-Candidate Matching

Input: {job_id: str, member_profiles:[{member_id, skills[], parsed_resume}]} Output: {matches:[{member_id, score:float, rationale:str}]}

61. Two-stage matching: (1) Skills overlap score = |intersection(job_skills, candidate_skills)| / |union(job_skills, candidate_skills)|. (2) Embedding similarity: embed job description and candidate summary using sentence-transformers (all-MiniLM-L6-v2). Combine: final_score = 0.6 * skill_overlap + 0.4 * embedding_similarity.
62. POST /ai/skills/match: Accept job_id + list of member profiles. Return ranked list with scores and rationale.
63. Edge case: If member has no skills and no resume text, score = 0.0 with rationale 'Insufficient profile data'.

Step 4: Hiring Assistant Supervisor Agent

The Supervisor consumes tasks from ai.requests, orchestrates skills, and emits results to ai.results. This is the multi-step Kafka-orchestrated workflow.

64. Kafka consumer: Subscribe to ai.requests topic. On new message with task_type='shortlist': (1) fetch all applications for job_id from Application Service (POST /applications/byJob via HTTP), (2) call Resume Parser for each applicant, (3) call Job-Candidate Matcher, (4) generate outreach draft for top N candidates, (5) emit result to ai.results.
65. Shared trace_id: The trace_id from the original task must propagate through every step and every Kafka message.
66. Step-by-step progress: After each step, emit intermediate message to ai.results with {trace_id, task_id, step:'parsing'|'matching'|'drafting', status:'running', partial_result:{...}}.
67. Final result: Emit {trace_id, task_id, step:'complete', status:'completed', result:{shortlist:[{member_id, score, rationale, outreach_draft}], metrics:{candidates_evaluated, top_score, avg_score}}}.

Step 5: REST Endpoints + WebSocket

68. POST /ai/request: Validate body {job_id, recruiter_id, task_type}. Generate task_id + trace_id. Store in MongoDB (status:pending). Produce to ai.requests topic. Return {task_id, trace_id}.
69. POST /ai/status: Return current task from MongoDB: {task_id, status, steps:[{step, status, timestamp, partial_result}], result?}.
70. POST /ai/approve: Store approval decision in MongoDB. If approved → trigger send (if outreach: call Messaging Service). Return {actioned: true}.
71. WebSocket /ai/stream/{task_id}: Client connects. Server keeps connection alive. Whenever ai.results Kafka consumer receives a message for this task_id, broadcast it to all connected WebSocket clients via ConnectionManager.

Step 6: Idempotency + Failure Handling

- Every Kafka message produced must include idempotency_key. Before processing, check MongoDB for existing task with same idempotency_key.
- LLM timeout: Set 30s timeout on all LLM calls. On timeout: mark step as failed, continue supervisor with partial results, emit ai.results with step_status:'failed', reason:'timeout'.
- Kafka produce failure: Retry 3x with exponential backoff. If all fail: mark task as failed in MongoDB, notify WebSocket clients.
- Consumer crash recovery: Store Kafka consumer offset only after successful MongoDB write.

PHASE 2: Career Coach + Evaluation (W5)

Step 7: Career Coach Agent (optional but graded)

POST /ai/coach: Input {member_id, target_job_id}. Fetches member profile + job description. Uses LLM to return: {resume_improvements:[], headline_suggestion:str, skills_to_add:[], rationale:str}. Stream progress via WebSocket.

Step 8: Evaluation Metrics (Required for Grading)

- **Matching quality:** Test with 20 sample job+candidate pairs. Manually label relevance (0/1). Compute Precision@5 (how many of top-5 results are relevant). Document in report.
- **Human-in-the-loop approval rate:** Track in MongoDB: count of tasks where action='approve' / 'edit' / 'reject'. Report percentages. Target demo: show at least 5 approval decisions.
- **Latency:** Log time from ai.requests produce to ai.results final emit. Report p50/p95 latency.

Deliverables Checklist

- FastAPI OpenAPI spec auto-generated at /docs — screenshot for submission
- Agent workflow diagram (Mermaid or draw.io) — for presentation
- Kafka topic design doc (ai.requests, ai.results payload samples)
- Skill descriptions doc: each skill's inputs, outputs, LLM prompt templates
- Evaluation metrics table: precision@5, approval rate, latency p50/p95

MEMBER 4: Kafka + Messaging + Connections Engineer

Domain: Kafka Infrastructure + Messaging Service + Connection Service

Phase 1 → Kafka Broker Setup, Messaging API, Connection API, Async Workflows | Phase 2 → Consumer Hardening, DLQ, Idempotency Validation, Failure Testing

GIVE THIS PROMPT + THE PROJECT DESCRIPTION DOC TO YOUR CODING AGENT.

Context & Scope

You own Kafka infrastructure and two domain services: Messaging and Connections. You are also responsible for the end-to-end async workflow — the path from UI → Kafka → consumer → DB → UI notification. You must ensure idempotency, DLQ provisioning, and consumer group design are correct for all topics used by all team members.

PHASE 1: Kafka + Services (W1–W4)

Step 1: Kafka Infrastructure Setup

72. docker-compose.yml already has Kafka + Zookeeper from Member 1. Your job: Create /infra/kafka_setup.py (or shell script) that creates all required topics with correct partition counts and replication factors.

```
# Required topics and configs:
TOPICS = [
  ('job.viewed',              partitions=3, replication=1),
  ('job.saved',               partitions=3, replication=1),
  ('application.submitted',   partitions=6, replication=1),
  ('application.status.updated', partitions=3, replication=1),
  ('message.sent',            partitions=6, replication=1),
  ('connection.requested',    partitions=3, replication=1),
  ('connection.accepted',     partitions=3, replication=1),
  ('ai.requests',             partitions=3, replication=1),
  ('ai.results',              partitions=3, replication=1),
  # Dead Letter Queues
  ('message.sent.dlq',        partitions=1, replication=1),
  ('application.submitted.dlq', partitions=1, replication=1),
]
# Consumer groups:
# analytics-consumer-group    → consumes: job.viewed, job.saved,
application.submitted
# messaging-consumer-group    → consumes: message.sent
# connection-consumer-group   → consumes: connection.requested, connection.accepted
# ai-supervisor-group         → consumes: ai.requests
# ai-results-group            → consumes: ai.results
```

73. Consumer group design: Each topic has exactly one consumer group per functional consumer. This ensures messages are not double-processed by the same service. Different services use different group IDs.
74. Test: Use kafka-topics.sh --list and kafka-consumer-groups.sh --describe to verify all topics and groups are created. Add a /health/kafka endpoint to the Messaging Service.

Step 2: Messaging Service (port 8004)

Build one endpoint at a time. Start with synchronous REST, add Kafka async in Step 3.

75. **POST /threads/open:** Check if thread already exists between participants (UNIQUE constraint on sorted participant pair). If yes, return existing thread_id (idempotent). If no, create new thread + add participants. Return {thread_id}.
76. **POST /threads/get:** Return thread metadata + participant list.
77. **POST /threads/byUser:** Join threads + thread_participants. Return threads with last message preview and unread count.
78. **POST /messages/list:** Return paginated messages for a thread, ordered by sent_at ASC. Include sender profile stub.
79. **POST /messages/send (synchronous baseline):** Validate thread exists + sender is a participant. Insert message to DB. Return {message_id}.

Step 3: Async Messaging via Kafka

80. Upgrade POST /messages/send to produce to message.sent topic before (or after) DB write. Use outbox pattern: write to DB first, then produce Kafka. If Kafka fails, message is still stored — outbox poller retries the Kafka produce.
81. Messaging consumer (messaging-consumer-group on message.sent): Receives event. Checks idempotency_key against processed_events table. If new: update thread.last_message_at. Mark as processed.
82. Failure simulation: Test what happens when Kafka broker is killed mid-send. Verify the outbox table has the message. Restart Kafka. Verify the outbox poller delivers it. Document this test.
83. DLQ: After 3 failed consumer retries, produce to message.sent.dlq. Log the reason. Never silently drop a message.

Step 4: Connection Service (port 8005)

84. **POST /connections/request:** Normalise pair: store as (min(a,b), max(a,b)) to prevent duplicates. Check: ALREADY_CONNECTED → 409 if status='accepted'. PENDING_REQUEST → 409 if pending request exists. Insert with status='pending'. Produce to connection.requested topic.
85. **POST /connections/accept:** Validate request exists + receiver_id matches authenticated user. Update status='accepted'. Increment connections_count on BOTH users (MySQL UPDATE members SET connections_count = connections_count + 1). Produce to connection.accepted.
86. **POST /connections/reject:** Update status='rejected'. No Kafka event needed.
87. **POST /connections/list:** JOIN connections with members to return connection profile stubs.
88. **POST /connections/mutual:** (extra credit) Return intersection of connection sets for two users. Use a WITH CTE or subquery.

Step 5: Application.submitted Consumer (Analytics bridge)

Member 1's Application Service produces to application.submitted. Your job: provision and test the consumer that forwards these events to the Analytics/Events Service.

89. Consumer (application-analytics-group on application.submitted): For each event, call POST http://localhost:8006/events/ingest with the full Kafka envelope as payload. Handle HTTP failures with 3 retries.
90. This bridges Kafka events to MongoDB analytics — critical for Member 5's dashboard.

PHASE 2: Hardening + Testing (W5–W6)

Step 6: End-to-End Async Flow Validation

Demonstrate this complete flow for grading:

91. UI sends POST /messages/send
92. Messaging Service inserts to DB + produces to message.sent
93. Consumer processes event → updates thread metadata in DB
94. AI Agent WebSocket broadcasts message_received event to connected frontend
95. Frontend updates UI without page refresh

Write a test script (test_async_flow.py) that runs this sequence and asserts each step completes within 2 seconds.

Step 7: Failure Mode Tests

- **Kill Kafka:** Verify POST /messages/send still returns 201 (DB write). Message goes to outbox. Restart Kafka. Verify delivery within 10s.
- **Kill Consumer:** Produce 10 messages. Kill consumer. Restart. Verify all 10 are processed (no duplicates, correct idempotency).
- **Duplicate event:** Produce same message twice with same idempotency_key. Verify only one DB write.

MEMBER 5: Analytics + Performance Engineer

Domain: Analytics Service + Redis Caching + JMeter Benchmarks

Phase 1 → Analytics Service, Redis Caching, Event Pipeline, Dashboards | Phase 2 → Performance Benchmarks (B/B+S/B+S+K), Scalability Report, Charts

GIVE THIS PROMPT + THE PROJECT DESCRIPTION DOC TO YOUR CODING AGENT.

Context & Scope

You own three critical parts: (1) the Analytics/Events Service that ingests Kafka events into MongoDB and serves dashboard data, (2) the Redis caching layer wired into Profile and Job services, and (3) the performance benchmark suite with JMeter. Your deliverables are directly graded: analytics graphs, Redis impact charts, and the 4-bar benchmark comparison.

PHASE 1: Analytics Service + Redis (W2–W4)

Step 1: Analytics/Events Service (port 8006)

96. **POST /events/ingest:** Accept Kafka event envelope. Validate required fields (event_type, trace_id, timestamp, actor_id, idempotency_key). Check idempotency_key in MongoDB events collection (unique index) — skip if duplicate, insert if new. Store full envelope. Return 202 accepted (async — do not wait for analytics computation).
97. **POST /analytics/jobs/top:** MongoDB aggregation: filter events by event_type IN [metric] and timestamp within window. Group by entity.entity_id (job_id). Count. Sort descending. Limit. JOIN with MySQL jobs table to get title/company. Return [{job_id, title, company, count}].
98. **POST /analytics/funnel:** For a job: count events by type in order: job.viewed → job.saved → application.submitted. Also track apply_start if frontend emits that event. Compute conversion rates between stages.
99. **POST /analytics/geo:** Aggregate application.submitted events by payload.member_location (requires member location to be in the Kafka payload). Group by city+state. Return sorted by count desc.
100. **POST /analytics/member/dashboard:** (1) Profile views: query MongoDB profile_views collection for member_id in last window_days, group by date. (2) Application status breakdown: query MySQL applications table GROUP BY status WHERE member_id = ?.

Step 2: Required Dashboard Data

Recruiter/Admin Dashboard data (all served by Analytics Service):

- Top 10 job postings by applications per month → /analytics/jobs/top with metric=applications
- City-wise applications per month for selected job → /analytics/geo
- Top 5 lowest-traction jobs → /analytics/jobs/top with metric=applications, sort ASC, limit=5
- Clicks per job posting → /analytics/jobs/top with metric=views
- Saved jobs per day/week → aggregate job.saved events grouped by date

Member Dashboard data:

- Profile views last 30 days → grouped by day
- Application status breakdown → count per status

Step 3: Redis Caching Layer

Wire Redis caching into Profile Service and Job Service. Coordinate with Member 1 who builds those services — they provide hooks, you implement the caching logic.

101. **Cache strategy:** Cache-aside pattern. On GET: check Redis first (key: member:{member_id} or job:{job_id}). Cache hit → return immediately. Cache miss → query MySQL → store in Redis with TTL=300s → return.
102. **Cache invalidation:** On UPDATE or DELETE: delete the Redis key immediately. Never serve stale data after a write.
103. **Job search cache:** Cache search results for up to 60s. Key: job:search:{hash(filters)}. Invalidate all job:search:* keys when any job is created/updated/closed.
104. **Redis key naming convention:** member:{id} · job:{id} · job:search:{query_hash} · recruiter:{id} · analytics:top_jobs:{window} · analytics:funnel:{job_id}:{window}
105. **Benchmark logging:** Add latency logging to every DB query and Redis lookup. Log: {operation, cache_hit:bool, latency_ms}. This data feeds your benchmark charts.

PHASE 2: Performance Benchmarks (W5)

Step 4: Seed 10,000 Records

106. Work with Member 6 (who owns the data pipeline) to ensure seed data is loaded: 10,000 members, 10,000 jobs, 10,000 recruiters, and ~50,000 applications (5 per member average).
107. Verify MySQL row counts before running benchmarks.

Step 5: JMeter Benchmark Suite

Define two benchmark scenarios:

- **Scenario A:** Job search + job detail view. Thread group: 100 concurrent users. Each user: POST /jobs/search (random keyword) → POST /jobs/get (random job_id from results). Ramp-up: 10s. Duration: 60s.
- **Scenario B:** Apply submit. Thread group: 100 concurrent users. Each user: POST /applications/submit (random member + job pair). Ramp-up: 10s. Duration: 60s.

Run each scenario in 4 configurations:

- **B (Base):** No Redis, no Kafka. Direct DB queries.
- **B+S (Base + SQL caching):** Redis enabled. Repeat identical search pattern to observe cache hit improvement.
- **B+S+K (+ Kafka):** Kafka enabled for async events. Measure apply latency difference.
- **B+S+K+Other:** Add connection pooling, query optimisation, or other techniques. Document what was added.

Step 6: Required Output Charts (for Presentation)

108. Bar chart: Scenario A — throughput (req/s) across B / B+S / B+S+K / B+S+K+Other. 100 concurrent users.
109. Bar chart: Scenario A — p95 latency (ms) across 4 configs.
110. Bar chart: Scenario B — throughput and p95 latency.
111. Redis impact chart: Cache hit rate % vs. DB query rate. Show the crossover point as cache warms up.
112. Deployment comparison: 1-service vs multi-replica latency and throughput (using AWS configs from Member 1).

All charts must be included in the presentation (Member 6 slots them in). Export as PNG.

Step 7: Analytics Dashboard Charts (for Submission)

Produce screenshots of all 7 required dashboard graphs with real seeded data:

- Top 10 jobs by applications (bar chart)
- City-wise applications for job X (horizontal bar)
- Top 5 lowest-traction jobs (bar chart)
- Clicks per job posting over time (line chart)
- Saved jobs per day/week (line chart)
- Member: Profile views last 30 days (line chart)
- Member: Application status breakdown (donut chart)

MEMBER 6: Data Engineering + Testing + Documentation

Domain: Dataset Pipeline + API Doc + Test Suite + Presentation

Phase 1 → Kaggle Dataset Pipeline, Seed Loader, API Document (Due 4/7), Repo README | Phase 2 → Test Class, Project Write-Up, Presentation Deck, Demo Script

GIVE THIS PROMPT + THE PROJECT DESCRIPTION DOC TO YOUR CODING AGENT.

Context & Scope

You own the data foundation (datasets + seed loader), the official API document deliverable (due April 7), the test suite, and the final presentation + write-up. You touch every other team member's work for documentation, testing, and integration validation.

PHASE 1: Data Pipeline + API Doc (W0–W2)

Step 1: CI Setup (Week 0 — after Member 1 creates the repo)

113. **Note:** Member 1 (Ayush) is the repo owner and handles initial repo creation + README. You take over from there.
114. Once repo is live, update README.md to add: dataset links (Kaggle jobs + resumes), data pipeline section, and 'How to seed data' instructions.
115. Create CONTRIBUTING.md with: branch naming convention, PR process, CI requirements, code style guide.
116. Set up GitHub Actions CI: .github/workflows/ci.yml — on PR: run lint (eslint for JS, flake8/ruff for Python) + unit tests per service. Fail PR if any check fails.

Step 2: Dataset Pipeline

117. **Download datasets (Kaggle API or manual):** LinkedIn Job 2023 dataset. Resume Dataset. Store raw CSVs in /data/raw/ (gitignored — add to .gitignore).
118. **Create /data/transform.py:** Reads raw CSVs. Maps columns to schema. Handles missing values (fill with defaults, not nulls for NOT NULL columns). Generates UUIDs. Outputs clean JSON files in /data/seeds/.
119. **Column mappings (jobs):** CSV: job_title → DB: title. company_name → company part of recruiter record. Skills extracted to job_skills table. Location parsed to city/state/country.
120. **Column mappings (resumes):** Category → skills list extraction. Resume_str → stored as resume_text in applications and MongoDB resumes collection.
121. **Synthetic data for missing fields:** Use Faker library to generate: phone numbers, headlines, about sections, salary ranges (uniform distribution $60k–$200k), connections_count (0–500). Generate 10,000 members, 10,000 recruiters, 10,000 jobs.
122. **Create /data/seed_loader.py:** Reads /data/seeds/ JSON files. Connects to MySQL + MongoDB. Inserts in batches of 500 (avoid timeout). Skips existing records (INSERT IGNORE or ON DUPLICATE KEY). Logs progress. Verifiable: prints row counts at end. Target: 10,000 members + 10,000 recruiters + 10,000 jobs + ~50,000 applications + 100,000 events in MongoDB.
123. **Verify:** Run SELECT COUNT(*) FROM members; SELECT COUNT(*) FROM jobs; — must show 10,000 each. Run db.events.countDocuments() — must show 100,000+.

Step 3: API Document (Deliverable A — Due April 7)

This is a graded deliverable. Create /docs/API_Document.pdf.

The API document must cover every endpoint in Part 3. For each endpoint include:

- Service name + base URL + port
- HTTP method + path
- Request body: field name, type, required/optional, description
- Response: success structure + all possible error codes with descriptions
- Kafka event emitted (if any): topic name + full envelope example
- Example curl command

Also include: DB schema diagrams (MySQL ERD from MySQL Workbench + MongoDB collection samples). Kafka topic table (from Part 4). Justification for MySQL vs MongoDB split. Architecture diagram (system-level, showing all services + connections). Agent workflow diagram.

PHASE 2: Tests + Write-Up + Presentation (W5–W6)

Step 4: Test Suite

Create /tests/ folder. One test file per service.

124. **Unit tests (pytest or Jest):** Profile Service: test create, get, update, delete, search. Test each validation error (missing required field, duplicate email). Test pagination.
125. Job Service: test create, get, search with filters, close. Test ALREADY_CLOSED error.
126. Application Service: test submit, duplicate detection (DUPLICATE_APPLICATION), closed job (JOB_CLOSED), status transitions (all valid + all invalid).
127. Messaging Service: test thread open (idempotent), message send, list.
128. Connection Service: test request, accept, reject, duplicate detection.
129. **Integration tests:** Test the full apply flow: create member → create job → submit application → update status → verify in byMember. Use a test DB (separate schema or Docker container).
130. **Failure mode tests:** Each failure mode in Part 5 must have a test that triggers it and asserts the correct HTTP status code and error code.
131. **Coverage target:** Minimum 70% line coverage on service-layer code. Run with pytest --cov or jest --coverage. Output report to /tests/coverage/.

Step 5: Project Write-Up (5 pages max)

Create /docs/Project_Writeup.pdf. Must cover:

- **Title page:** Group number, team name, member names + roles.
- **Contributions page:** One short paragraph per member describing their specific contribution.
- **Object management policy:** How entities are created, updated, soft-deleted vs hard-deleted. Why UUID primary keys. Reference counting for connections_count.
- **Heavyweight resources:** How PDF resumes are handled (stored as URL reference, actual file in S3/object storage, not in DB). How MongoDB handles large event collections (capped? TTL indexes?).
- **Write policy:** When data is written to DB (immediately on mutation, not on read). Cache invalidation strategy (delete-on-write, not update-on-write). Why outbox pattern for Kafka ensures no lost events.
- **Screen captures:** UI showing real data (Member 2 provides). DB schema screenshot (MySQL Workbench). Analytics dashboard screenshot (Member 5 provides).
- **Observations and lessons learned:** 1 page max. What would you do differently? What surprised you about Kafka? Distributed systems challenges encountered.

Step 6: Presentation Deck (Due April 28)

Create a slide deck (Google Slides or PowerPoint) covering all required presentation sections:

132. Slide 1: Group number + team details + member table
133. Slide 2: Database Schema — MySQL ERD + MongoDB collection samples
134. Slide 3: System Architecture Diagram — 3-tier with all services, Kafka in center, databases
135. Slides 4–5: Agent Architecture — workflow diagram, Kafka topic design for AI, skill descriptions
136. Slides 6–7: Performance — 4 bar charts (B/B+S/B+S+K/B+S+K+Other) for Scenario A and B. Redis impact chart. Deployment comparison.
137. Slide 8: Agent Metrics — precision@5 matching quality, approval rate, latency p50/p95
138. Slide 9: Analytics Dashboard — screenshots of all 7 graphs with real data
139. Slide 10: Lessons Learned + What Would We Do Differently?

Step 7: Demo Script (Due May 5)

Prepare a 10-minute live demo script covering all 3 required demo flows:

140. **Flow 1 (Member):** Register member → complete profile → search jobs → view job details → apply → track application status in My Applications.
141. **Flow 2 (Recruiter):** Login as recruiter → create job posting → view applicants → update application status → view analytics dashboard.
142. **Flow 3 (AI Copilot):** Recruiter clicks 'AI Shortlist' on a job → WebSocket streams progress → AI returns shortlist + outreach draft → recruiter approves one → message sent.

Each flow must show real data (seeded 10,000 records). Prepare a fallback video recording in case of live demo issues.

PART 6 — INTEGRATION CHECKLIST & MERGING PROTOCOL

Weekly Integration Points

Week	Gate Requirement	Who Tests	Definition of Done
W1	docker-compose up → all services start + healthchecks pass	All	GET /health on all 7 services returns 200
W2 (Apr 7)	API Document submitted + all endpoints stubbed (return mock data)	Member 6	Every endpoint returns correct HTTP code even if data is fake
W3	Flow 1 works end-to-end: register → search jobs → apply	Member 2 + Member 1	Application in DB, Kafka event emitted
W4	Kafka async flow: apply → Kafka → consumer → DB + UI notification	Member 4 + Member 2	WebSocket push received in browser
W5	Analytics dashboard shows real data. JMeter benchmarks run.	Member 5	All 7 charts populated. 4-bar benchmark chart ready
W6	Full system on AWS. Load test passes. No crashes.	Member 1 + Member 5	300 req/s sustained for 60s without error rate > 1%
W6 (May 5)	Demo script runs without errors. All 3 flows complete.	All	Presentation submitted, demo recorded as backup

Integration Rules

- **Never break main:** All PRs must pass CI. If your feature is not ready, use a feature flag — don't merge broken code.
- **API contract is frozen:** After April 7, no endpoint path or response schema changes without team-wide discussion and update to this document.
- **Port contract is frozen:** The port map in Part 1 is permanent. Never change a port without updating all team members.
- **Kafka envelope is frozen:** After April 7, the Kafka event envelope in Part 4 is permanent.
- **Weekly sync:** 30-minute team standup every Monday. Each member: what did I finish, what am I doing this week, what is blocking me.
- **Shared Postman collection:** Member 6 maintains /docs/LinkedInClone.postman_collection.json. Add one request per endpoint as it's implemented. All members can test any service.

END OF DOCUMENT — THIS IS THE SINGLE SOURCE OF TRUTH

Last updated: March 2025 | Any changes must be communicated to all 6 members